Dimensionality reduction and unsupervised learning

Olivetti faces · four hundred photographs, no labels

LECTURE 8 | GÉRON CH. 7 & 8

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Seven lectures, and every one of them was handed the answers:

Lecture	What the data came with
1–2 · California housing	a price for every district
3 · MNIST	a digit for every image
4–5 · Titanic	a survivor flag for every passenger
6–7 · CoverType	a cover type for every plot
8 · Olivetti faces	**✗ nothing**

Last lecture averaged many trees: the variance of an average of correlated predictors falls to $\rho\sigma^2$ and no lower, which is why bagging, random forests and extra-trees all work at lowering the correlation between their members — and boosting, built sequentially, attacks the bias instead. Every one of those models was scored against a label.

Today there is no target column. That changes what a metric can be, what a baseline is, and how you find out you were wrong.

Today

1. Four hundred points in four thousand dimensions, and what that costs
2. **The mathematics** — PCA as the SVD of the centred data, in seven lines
3. **Reducing the dimension** — eigenfaces, reconstruction, and a reducer fitted before the split
4. **Clustering** — k-means, and how to choose k without labels
5. Beyond k-means: DBSCAN, Gaussian mixtures, anomalies, and where to spend forty labels

Chapters 7 and 8 together, because the second one does not work on this data until the first one has been applied to it.

Open Lecture 8 in Colab

The brief

> **THE REQUEST**
>
> “We have a photographic archive. We do not know how many distinct people are in it. Group the photographs by person, so that a human can name a *group* instead of naming every photograph.”

- The archive is not annotated, and nobody remembers who is in it
- A trained annotator costs money and time per photograph
- We are funded for **forty** labels. That is all

This is the ordinary situation outside a benchmark: data is cheap, labels are not.

Why this is a different kind of problem

Supervised

- You are given $(\mathbf{x}^{(i)}, y^{(i)})$
- The metric compares prediction to truth
- The baseline is “predict the majority”
- You know when you are wrong

Unsupervised

- You are given $\mathbf{x}^{(i)}$ only
- The metric compares the grouping to *itself*
- The baseline is “group at random”
- You find out you were wrong when someone looks

THE CONSEQUENCE FOR THE REST OF THE LECTURE

Every number computed today is a statement about **geometry**, not about people. Whether the geometry *is* the people is a separate question, and it is answered by looking.

The corpus, and a photograph as a point

Quantity	Value
Photographs	400
Distinct people	40
Photographs per person	10
Pixels per photograph	4,096
The whole corpus in memory	6.6 MB

$$\mathbf{x}^{(i)} \in \mathbb{R}^{4096}$$

$$x_j \in [0, 1]$$

Flatten the 64 × 64 grid and a photograph is a vector. Every method today measures a distance between two of them.

✗ **Ten times more dimensions than points. Four hundred points cannot fill four thousand dimensions, and the first block of this lecture is about what follows from that.**

Forty people

One photograph of each of the 40 people. The corpus holds 400, and none of them arrives labelled



Same crop, same lighting rig, same background. What varies between rows is the person — and, as much, the lamp.

`y` exists. The brief does not have it

Olivetti is a benchmark, so it ships with the identity of every photograph. The archive we are pretending to hold does not.

> **THE RULE, KEPT FOR THE WHOLE LECTURE**
>
> `y` is used in exactly two ways, both announced on the slide where they happen: as the *forty labels we paid for*, and as an **audit**, to find out what those forty could not have told us.

Any other use of `y` is the same error as scoring on the test set, wearing different clothes. Slides marked *audit* are results the project itself could not have computed.

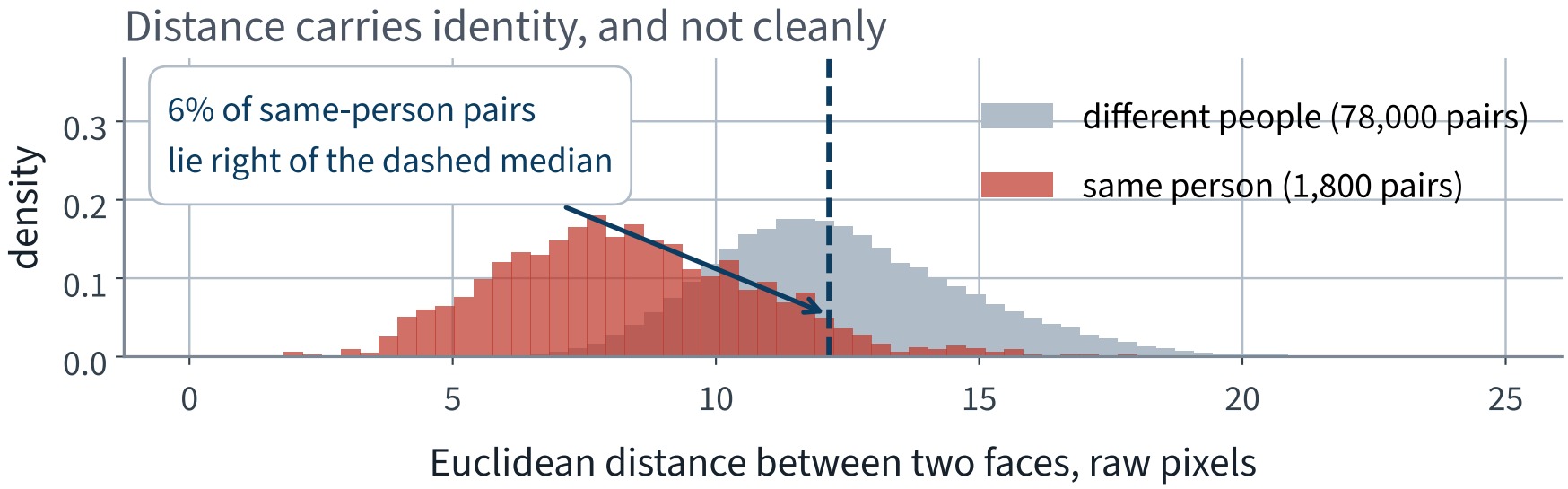
Before anything is fitted: can this work at all?

k-means groups points that are close in Euclidean distance. So one measurement decides whether the rest of the lecture can work (an audit — it uses the hidden labels):

Are two photographs of the same person closer than two photographs of different people?

Euclidean distance, raw pixels	Pairs	Mean	Median
Same person	1,800	8.38	8.15
Different people	78,000	12.41	12.15

The gap is real, and it is small



✗ 6% of same-person pairs sit further apart than the *median* different-person pair. Pixel distance is partly identity and partly lighting.

And it is not free to work there

One model-selection sweep — k-means at every k from 2 to 60, ten restarts each, scored by silhouette:

Step	Seconds	Cost model
590 runs of Lloyd's algorithm	77	$O(\text{iterations} \times m\,k\,n)$
59 silhouette scores	2	$O(m^2 n)$
One sweep, total	✗ 78	two threads, one machine

Both cost models are **linear in n**, the number of features. Two of the three factors are properties of the problem. The third is a property of the *representation*.

✓ So: how many of those 4,096 numbers are doing work — and what exactly do you lose by throwing the rest away?

The question, and its notation

Among all d-dimensional subspaces, which one loses the least when you project the data onto it — and *how much* does it lose?

- $\mathbf{X} \in \mathbb{R}^{m \times n}$ — one photograph per *row*: $m = 400$ rows, $n = 4096$ columns
- $\mathbf{W} \in \mathbb{R}^{n \times d}$ with $\mathbf{W}^\intercal \mathbf{W} = \mathbf{I}_d$ — an orthonormal basis of the subspace we are choosing
- $\|\mathbf{A}\|_F^2 = \sum_{ij} a_{ij}^2 = \operatorname{tr}(\mathbf{A}^\intercal \mathbf{A})$

> **WHAT THE ANSWER WILL BE**
>
> A theorem about the optimal subspace that also names the price exactly, before you choose d. Everything below is linear algebra; there is no probability in it and none is needed.

Centre first, and say why

$$\tilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}\bar{\mathbf{x}}^{\mathsf{T}}$$

$\bar{\mathbf{x}}$ holds the column means, the mean face. Without centring, the direction of largest $\sum_i \|\mathbf{w}^{\mathsf{T}}\mathbf{x}^{(i)}\|^2$ is essentially $\bar{\mathbf{x}}$ itself, and the first component describes nothing but “this is a photograph of a face”.

WHEN THIS BITES

Scikit-Learn’s `PCA` centres for you. If you write the SVD by hand it does not, and the result is a plausible-looking first component and no error message. It is the commonest silent bug in a hand-rolled PCA.

The derivation, in seven lines

Reproduce this from the statement of the problem.

EXAMINABLE

1. $E(\mathbf{W}) = \|\tilde{\mathbf{X}} - \tilde{\mathbf{X}}\mathbf{W}\mathbf{W}^\intercal\|_F^2$
   the objective: squared distance to the subspace

2. $E(\mathbf{W}) = \|\tilde{\mathbf{X}}\|_F^2 - \|\tilde{\mathbf{X}}\mathbf{W}\|_F^2$
   Pythagoras; the first term is free of $\mathbf{W}$

3. $\|\tilde{\mathbf{X}}\mathbf{W}\|_F^2 = \operatorname{tr}(\mathbf{W}^\intercal\mathbf{C}\mathbf{W})$
   the Frobenius norm as a trace, with $\mathbf{C} = \tilde{\mathbf{X}}^\intercal\tilde{\mathbf{X}}$

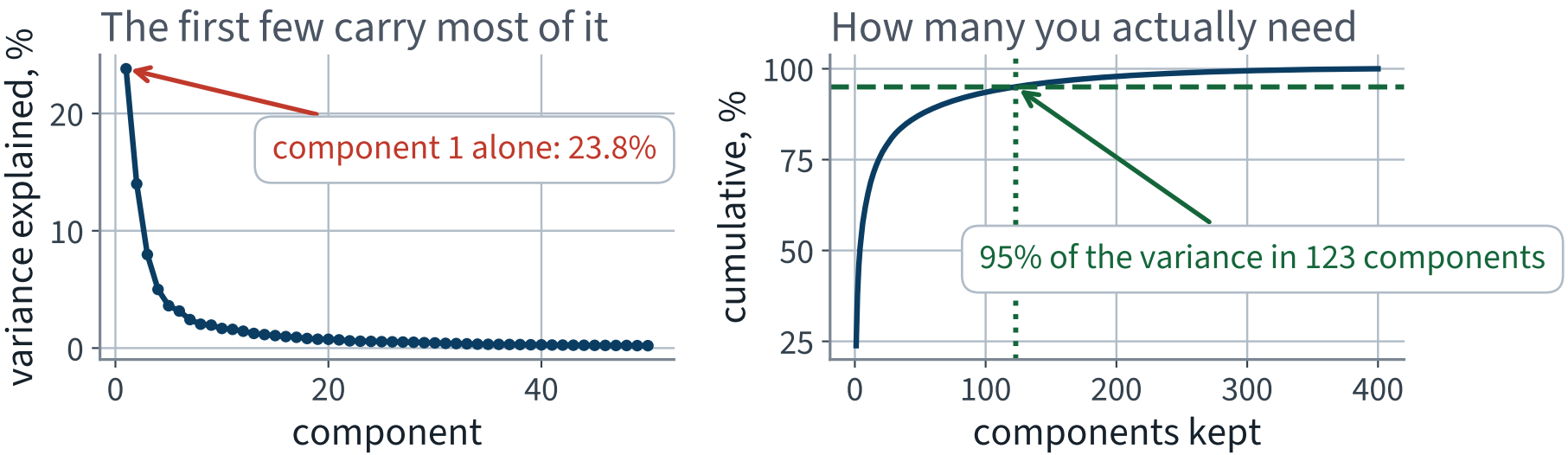
4. $\mathbf{C} = \mathbf{V}\mathbf{\Sigma}^2\mathbf{V}^\intercal$
   from the SVD $\tilde{\mathbf{X}} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\intercal$

5. $\operatorname{tr}(\mathbf{W}^\intercal\mathbf{C}\mathbf{W}) = \sum_j \sigma_j^2 h_j$, with $0 \le h_j \le 1$ and $\sum_j h_j = d$
   write $\mathbf{W} = \mathbf{V}\mathbf{A}$; h_j is a squared row norm of $\mathbf{A}$

6. the maximum is $\sum_{j \le d} \sigma_j^2$, at $\mathbf{W} = \mathbf{V}_d$
   spend all d units of weight on the largest σ_j^2

7. $\min_{\mathbf{W}} E(\mathbf{W}) = \sum_{j > d} \sigma_j^2$
   the error PCA leaves is exactly the tail of the spectrum

How many components you actually need



The explained variance ratio $r_j = \sigma_j^2 / \sum_i \sigma_i^2$ is line 7 normalised: keeping d components removes exactly $\sum_{j \le d} r_j$ of the squared error. 123 components hold 95% of the variance of 4,096 pixels — a factor of 33 fewer numbers per photograph.

The theorem, in one line of code

```python
from sklearn.decomposition import PCA

Z = PCA(n_components=0.95, random_state=42).fit_transform(X)
print(Z.shape)                                  # (400, 123)
```

`n_components=0.95` — a float between 0 and 1 is read as a **variance target** rather than a component count, and scikit-learn chooses d for you from the cumulative ratio. Here it chooses 123.

✓ An integer means “this many components”; a float means “this much variance”. One character apart, and `n_components=1` and `n_components=1.0` do completely different things.

The mean face, then the first fifteen components

The mean face, then the first 15 principal components. Each component is itself an image



✗ A component is a vector in $\mathbb{R}^{4096}$, so it is an image — an ***eigenface***. Look at the first few. They are lighting — which is exactly what the distance histogram predicted would dominate.

The same held-out face, with components taken away

One held-out face, rebuilt from this many components



Recognisable well before 123 components. Identity survives a compression that the pixel values do not — and each PCA here was fitted on the training faces only.

One procedural condition, because it is invisible

A reducer is an estimator: it is **fitted**. Fit it before the split and the held-out faces have helped choose the subspace they are then judged in.

```
pca = PCA(n_components=0.95).fit(X)          # all 400 photographs
Z   = pca.transform(X)

Z_tr, Z_te, y_tr, y_te = train_test_split(   # the split comes after
    Z, y, test_size=0.3, stratify=y, random_state=42)
```

The 280 training faces, once centred, span at most a 279-dimensional subspace of $\mathbb{R}^{4096}$, so adding 120 more points genuinely changes which directions survive. This is not a rounding error.

What it costs, over twenty splits

$d = 123$, 20 stratified splits	Fitted on train	Fitted on everything	Leaky wins
Reconstruction error, held out	0.00243	0.00096	✗ 20 of 20
Accuracy, held out (logistic regression on the components)	97.4%	97.4%	3 of 20

The leak makes the reconstruction error look **61%** better and moves the accuracy by nothing measurable.

THE DECISION RULE, AND WHY IT IS STILL PROCEDURAL

Fitting an unsupervised step on everything costs you **exactly when that step's own objective is the number you report**. Reconstruction error is that objective; downstream accuracy is not — and you cannot tell which row you are in without doing the split. So: `Pipeline([("pca", PCA(0.95)), ("clf", …)])`, which refits the PCA inside every fold and makes the leak structurally impossible rather than merely avoided.

k-means, in five lines

1. Choose k centroids (scikit-learn uses *k-means++*, which spreads the initial ones out)
2. Assign every point to its nearest centroid
3. Move every centroid to the mean of its assigned points
4. Repeat 2–3 until no assignment changes
5. Repeat the whole thing `n_init` times and keep the best

> Step 5 is not optional. Lloyd's algorithm converges — to a local minimum that depends entirely on where you started. `n_init` changed its default between scikit-learn versions, so name it.

What k-means assumes, whether you asked or not

FOUR ASSUMPTIONS, THREE OF THEM FALSE HERE

Convexity. The partition is a Voronoi diagram of the centroids, so every cluster is convex.

Spherical, equal-sized. The only quantity is distance to a centre.

Total assignment. Every point belongs to exactly one cluster; there is no “none of these”.

k is given. Nothing in the algorithm chooses it.

The first three are dropped by DBSCAN and by Gaussian mixtures, in the last block of this lecture. The fourth — choosing k — is what the rest of this section is about.

Inertia, and why it cannot choose k

$$J(k) = \sum_{i=1}^{m} \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}_{c(i)} \right\|_2^2$$

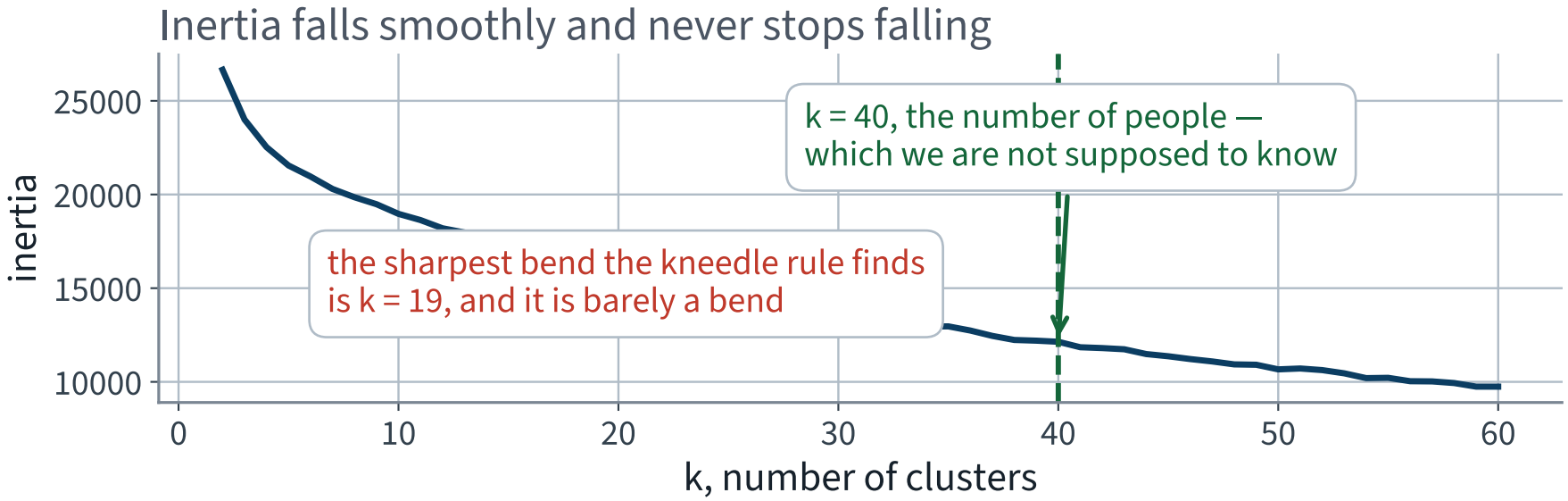
THE SUM OF SQUARED DISTANCES TO THE ASSIGNED CENTROID — WHAT K-MEANS MINIMISES

A PROPERTY, NOT AN ACCIDENT

J is **non-increasing in k** — refining a partition cannot increase the sum of squared distances to the assigned centre — and $J(m) = 0$ exactly. There is no interior minimum to find, at any k, on any dataset.

So “minimise inertia” selects one cluster per photograph. The elbow rule exists to work around that.

Find the elbow



A smooth convex curve. The *kneedle* rule — furthest below the chord, both axes rescaled — picks $k = 19$; the archive has 40, and 46% of the inertia at $k = 2$ is still present at $k = 40$.

A criterion that *does* have an interior maximum

✗ The elbow is not a criterion. It is a picture of a criterion that does not exist, and no better heuristic repairs it, because the information is not in J. So use one that does.

For a point i, let a be its mean distance to the other points of *its own* cluster, and b its mean distance to the points of the *nearest other* cluster.

$$s_i = \frac{b - a}{\max(a, b)} \in [-1, 1]$$

- $s_i \to +1$: far inside its own cluster · $s_i \approx 0$: on a boundary · $s_i \to -1$: assigned to the *wrong* cluster
- The **silhouette score** is the mean of s_i. It is penalised for splitting a group *and* for merging two, so unlike inertia it has an interior maximum
- It costs $O(m^2 n)$ — every pair, over every feature. Both factors matter: the archive is bigger than 400, and n is larger than it needs to be

The number the stakeholder reads: adjusted Rand index

Compare two groupings pair by pair, and count the pairs they agree about — together in both, or apart in both:

$$\mathrm{RI} = \frac{a + b}{\binom{m}{2}}, \qquad \mathrm{ARI} = \frac{\mathrm{RI} - \mathbb{E}[\mathrm{RI}]}{1 - \mathbb{E}[\mathrm{RI}]}$$

RI is high for free at large k — almost every pair is apart in both groupings. The *adjusted* index subtracts what chance would give, so random scores 0 in expectation at any k, perfect scores 1, and negative is worse than chance.

✗ We can compute it on the forty labelled photographs only: 780 pairs, of which just 22 are the same person. Treat it as a sanity check with a wide interval, never as a ranking.

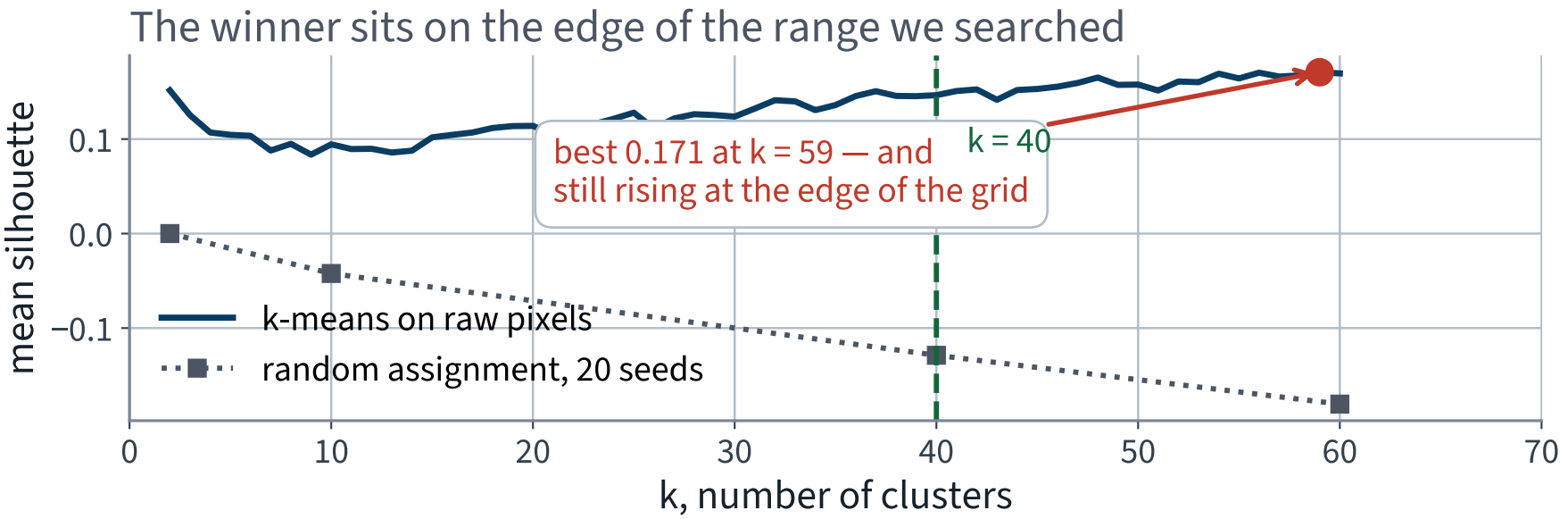
Before reading any silhouette: what does *nothing* score?

Put every photograph in a uniformly random group, twenty times, and look at the spread. Both metrics have no natural scale, and a number with no null has no units.

Random assignment	Silhouette	ARI on the forty
$k = 10$	−0.0422 ± 0.0045	−0.0040 ± 0.0205
$k = 40$	−0.1288 ± 0.0107	−0.0089 ± 0.0244
$k = 60$	−0.1805 ± 0.0124	−0.0075 ± 0.0261

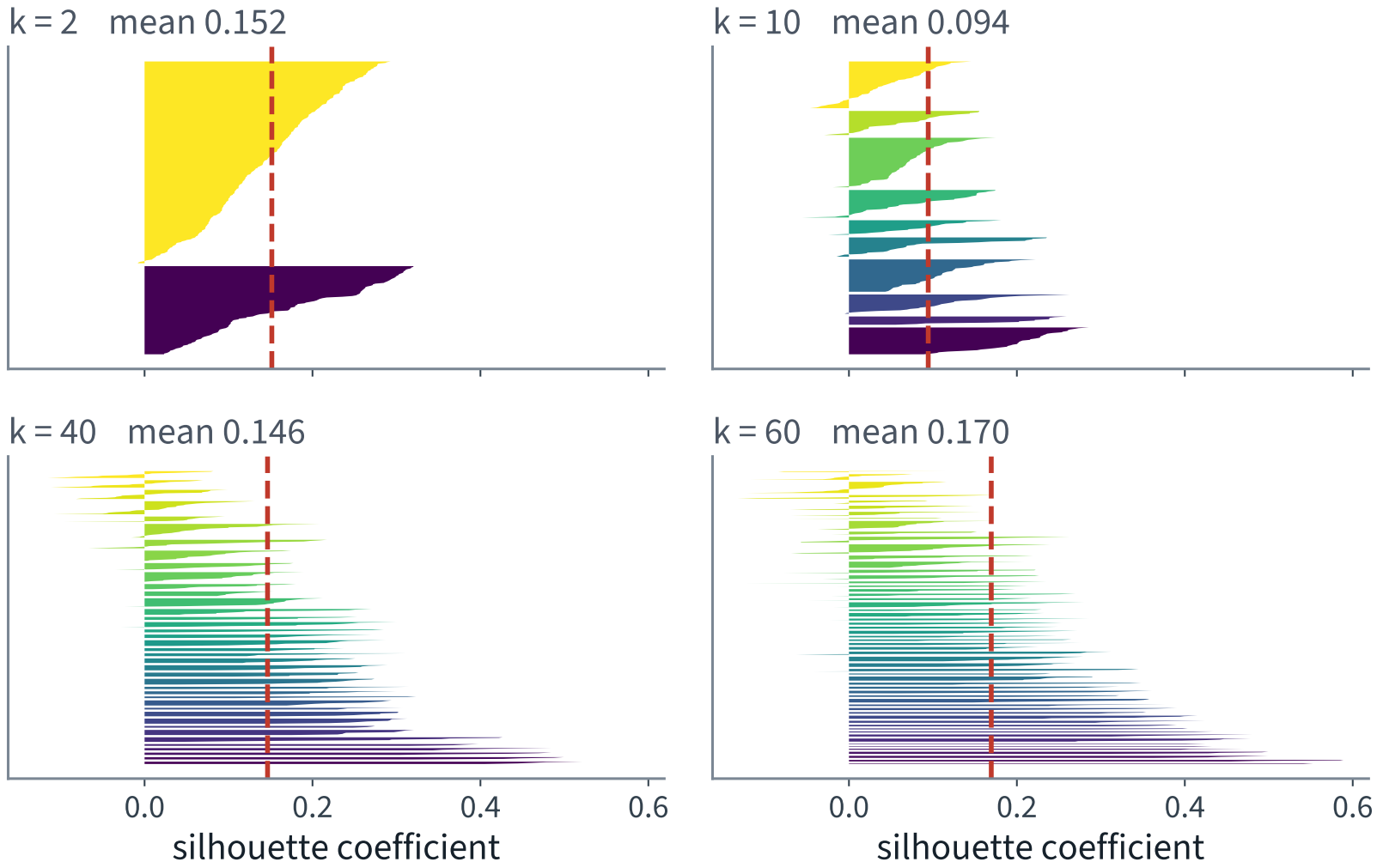
✓ Both anchors sit at zero, and now you know the width of the noise around zero. A silhouette of 0.02 is not a discovery.

Silhouette against k, read against the anchor



A maximum exists. It is also low, and the curve near it is nearly flat — so the chosen k is not decisively better than its neighbours.

The mean hides the shape



One knife per cluster: width is size, length is quality. At larger k they are short and uneven.

A mean silhouette means nothing until you look

cleanest cluster at k = 40: 10 photographs, 1 identity, purity 1.00



worst cluster at k = 40: 24 photographs, 10 identities, purity 0.21



✗ **The cleanest and the worst cluster at $k = 40$. The bottom row is not a person. It is a *lighting condition* — several different people lit and posed the same way.**

Counting it, rather than describing it

At $k = 40$, against the hidden identities	Value
Cleanest cluster: 10 photographs, purity	**1.00 ✓**
Worst cluster: 24 photographs of 10 people, purity	**✗ 0.21**
Mean purity across the forty clusters	0.74
Clusters holding exactly one person	16
Clusters with exactly ten members	7
Cluster sizes, smallest to largest	4 to 25

Every true group has exactly ten members. The overlap the distance histogram showed before anything was fitted — pixel distance is partly identity and partly lighting — is now a count.

Now put the two halves of the lecture together

```
Z = PCA(n_components=0.95, random_state=42).fit_transform(X)   # 123

for k in range(2, 61):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(Z)
    sil.append(silhouette_score(Z, km.labels_))
```

The clustering code is unchanged. The grid is unchanged, the seed is unchanged, `n_init` is unchanged. Only *what k-means is looking at* changed.

✓ One line, and both cost models lose a factor of 33 in their n.

DBSCAN, in three definitions

- A point is a **core point** if at least `min_samples` points lie within `eps` of it
- Core points within `eps` of each other are in the same cluster; so are the non-core points in their neighbourhoods
- Everything else is **noise**, labelled -1

```
db = DBSCAN(eps=6.0, min_samples=3).fit(Z)
n_clusters = len(set(db.labels_) - {-1})     # exclude the noise label
```

`DBSCAN` has no `predict`. It is transductive: to label a new point you fit again, or you train a classifier on its output.

Gaussian mixtures

Model the data as coming from k Gaussians with unknown means, covariances and weights, fitted by expectation–maximisation:

$$p(\mathbf{x}) = \sum_{j=1}^{k} \pi_j \, \mathcal{N}\left(\mathbf{x};\, \boldsymbol{\mu}_j,\, \boldsymbol{\Sigma}_j\right)$$

- Clusters are ellipsoids, so they may be elongated and of different sizes
- Assignment is *soft*: `predict_proba` returns a distribution over components
- It is a **density model**, so it also scores how likely a new point is — which is anomaly detection, shortly

Anomaly detection, two ways

By density

A Gaussian mixture is a density. Score every image and call the lowest few anomalous.

```
d   = gmm.score_samples(Z)
bad = d < np.percentile(d, 3)
```

By reconstruction error

An image the eigenfaces cannot rebuild is unlike the images that built them.

```
R   = pca.inverse_transform(pca.transform(Xa)
err = ((R - Xa) ** 2).mean(axis=1)
```

Test both by planting twelve corrupted images — four rotated, four dimmed, four double-exposed — and counting how many each puts in its top twelve. The notebook does exactly that, with both detectors fitted on the clean corpus only.

Finally: the forty labels

Split the corpus 280 / 120, stratified by identity. We may label forty of the 280 training photographs. Where we spend them turns out to matter more than what we do with them.

1. Forty **at random**
2. Forty chosen as the photograph nearest each of forty cluster centroids — `km.transform(Z).argmin(axis=0)`
3. The representative's label **propagated** to its whole cluster — 280 labels from 40
4. Propagated only to the 75% of each cluster closest to its centroid
5. All 280 true labels — the ceiling, which the project cannot buy

Same classifier, same held-out faces, twenty splits. The only thing that varies is *which* forty were labelled.

The numbers

Labels used	Held-out accuracy	sd over 20 splits
40 at random	48.1%	3.8%
40, one per cluster	**60.5% ✓**	4.8%
Propagated to the whole cluster	55.6%	4.8%
Propagated to the closest 75%	55.1%	4.9%
All 280 true labels	**✓ 97.4%**	1.6%

Twenty splits, because the differences between the middle three rows are of the same order as their spread — and a comparison that turns on less than the spread is not a comparison.

Propagation buys 280 labels from 40 and does *worse* than the 40 alone, because a wrong propagated label is worse than no label. Report that, do not bury it.

The diagnosis was right. The obvious repair still fails

Lighting dominates the distance. Two one-line repairs, on the same 400 faces, choosing k by silhouette and reporting ARI exactly as before:

Representation	Best k	Silhouette	ARI, all 400 (audit)
PCA to 123 dimensions	60	0.205	**0.524 ✓**
PCA 123, first 3 components discarded	60	0.184	**✗ 0.508**
Unit-norm each image, then PCA 123	58	0.189	**✗ 0.464**

Neither helps. The leading components carry pose and face shape as well as illumination, so removing them throws away signal with the nuisance. The remedy that does work is a representation learned for the task rather than for variance — which is what Part VI builds, and why a pretrained encoder beats a hand-chosen projection.

✗ Note also what chose k. Sweeping k and keeping the best *ARI* makes both repairs look like gains — because ARI is computed from the labels this whole lecture is pretending not to have.

The notebook for this lecture

Open Lecture 8 in Colab

- **Runs on free CPU**, top to bottom
- PCA by hand from the SVD of the centred data, checked against scikit-learn to 10^{-4}; the explained variance ratio, the eigenfaces and the reconstructions
- Everything else on today's slides — the distance histogram, the leak, the sweep in raw pixels and compressed, the elbow, the random anchor, the silhouette diagrams, DBSCAN, the anomalies, the forty labels and the lighting repairs

> **WHAT TO CHANGE, AND WHAT SHOULD HAPPEN**
>
> Every specification box carries a **try** line. The cheapest one: change `n_components=0.95` to `0.99`, which takes d from 123 to 260. The sweep slows by roughly the ratio of the two, and the ARI barely moves — which is the whole trade-off in one edit.

What to remember

1. PCA is the SVD of the **centred** data. The principal subspace minimises squared reconstruction error exactly, and the error it leaves is $\sum_{j>d} \sigma_j^2$ — the tail of the spectrum, readable before you choose d
2. Minimising that error and maximising the projected variance are the same problem, by Pythagoras. The explained variance ratio is that theorem normalised, and it is how d is chosen: 123 components hold 95% of 4,096 pixels
3. A reducer is fitted, so it is fitted after the split: the leak flatters its own objective, the reconstruction error, and not the accuracy
4. Inertia cannot choose k: it is monotone in k and zero at $k = m$. The silhouette can, at $O(m^2 n)$, and must be read against a random anchor — and then against the faces, because a mean hides a cluster that is a lighting condition rather than a person
5. k-means assumes convex, equal-sized clusters, total assignment and a known k. DBSCAN finds dense regions and a noise label; a Gaussian mixture fits ellipsoids and is a density, so it can also flag anomalies — as can the PCA reconstruction error
6. Forty labels spent on cluster representatives are worth a quarter more accuracy than forty spent at random. Which forty mattered more than what was done with them

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 8

five, in the style of the written paper

Exercises — Lecture 8

1 PCA is derived by maximising $\|XW\|_F^2$ subject to $W^\top W = I$. State why the data must be centred first, and what the first component becomes if it is not. [5]

2 The explained variance ratio is sometimes called a heuristic for choosing d. Using the result of the PCA derivation, say why it is not one, and what keeping 95% of the variance leaves as reconstruction error. [5]

3 A pipeline fits PCA on the whole dataset and then splits. Name what leaked, and say which held-out number the leak flatters and which it leaves alone. [4]

Solutions on Lecture 9's deck.

Exercises — Lecture 8 (continued)

4 Inertia always falls as k rises, so it cannot choose k. State what silhouette adds, and one situation where it also fails. [5]

5 You have 4,096-dimensional face vectors and are told two faces are “close”. Explain why that alone does not mean they are the same person. [4]

Solutions on Lecture 9’s deck.

THE FIVE SET AT THE END OF LECTURE 7

Solutions Lecture 7

not part of this lecture

Solutions — Lecture 7

1. For n predictors each of variance σ^2 and pairwise correlation ρ, the variance of their average…

✓ It tends to $\rho\sigma^2$ — correlation, not ensemble size, sets the floor.

- The second term vanishes; the first does not depend on n at all
- Adding members past a point buys nothing, which is why the design effort goes into *decorrelating* them

2. Bagging measures $\rho = 0.072$ with one member at 78.8% accuracy; the random forest measures $\rho = 0.043$…

✓ Lowering ρ is not free. The forest lowers it by letting each split see only 7 of 54 columns, which handicaps every member, and the formula is about variance alone: σ^2 sits in both terms and bias is not in it at all.

- $\mathrm{Var}(\bar{f}) = \rho\sigma^2 + (1-\rho)\sigma^2/n$ predicts the variance of the average; accuracy also depends on how good each member is
- On this dataset the decorrelation is worth less than the weaker members cost: 89.8% for bagging against 88.4% for the forest

Solutions — Lecture 7 (2)

3. You set bootstrap=False and oob_score=True. State what happens and why.

✓ It raises: with no bootstrap there are no out-of-bag rows.

- Out-of-bag scoring uses the rows a member did not sample
- Without sampling, every member sees every row and the estimate is undefined

4. Impurity importance ranks a column of random numbers tenth of 55; permutation importance on held-out data…

✓ Shuffle one column of the held-out set, re-score the fitted model, and report the fall in accuracy, repeated for a spread. A random column carries no information the model can use on new rows, so shuffling it costs nothing.

- Impurity importance sums the impurity decrease of every training-set split on a column, so it is earned on the rows the trees were grown on
- Permutation importance is measured on rows the model never saw: only a column whose values matter for those predictions loses accuracy when they are scrambled

Solutions — Lecture 7 (3)

5. A forest improves accuracy over one tree by far less than the variance reduction would suggest. Is this a…

✓ No — variance is only one term of the error.

- Averaging reduces variance and leaves bias untouched
- If the remaining error is mostly bias or noise, removing variance moves the accuracy very little

LECTURE 9 · GÉRON CH. 9

Neural networks

Back to labels: ten classes of small greyscale images, and a pixel as a feature
The multilayer perceptron, and why a stack of linear layers is a linear model
A layer in matrices, softmax and cross-entropy, and the parameter count before the fit
A network fitted and tuned by hand — and the four walls `fit()` puts round you

Run the Lecture 8 notebook first